

Cheerio & Node Projects

AIT 618

Cheerio

What is Cheerio?

- A nifty Node package that allows developers to scrape front-end web pages
 - Use 'npm install cheerio'
 - Use `var cheerio = require('cheerio');`
- What are the risks with scraping a front-end web page?

How does Cheerio Work?

- Cheerio will allow a developer to iterate through HTML elements using DOM
- Developers can access specific HTML elements on a page or even loop through elements
 - I.e. if there is a webpage that has a <table>, it is possible to loop through all <td> tags to find a specific value that is of interest
- Cheerio is mainly written in jQuery, but I will show the necessary jQuery components needed

Building a Simple Web Scraper

- What is needed
 - The cheerio module
 - The request module
- Request is needed to perform a GET call on the on the webpage
 - The GET will pull the raw HTML so it can be stored in a variable for cheerio to be able to search through

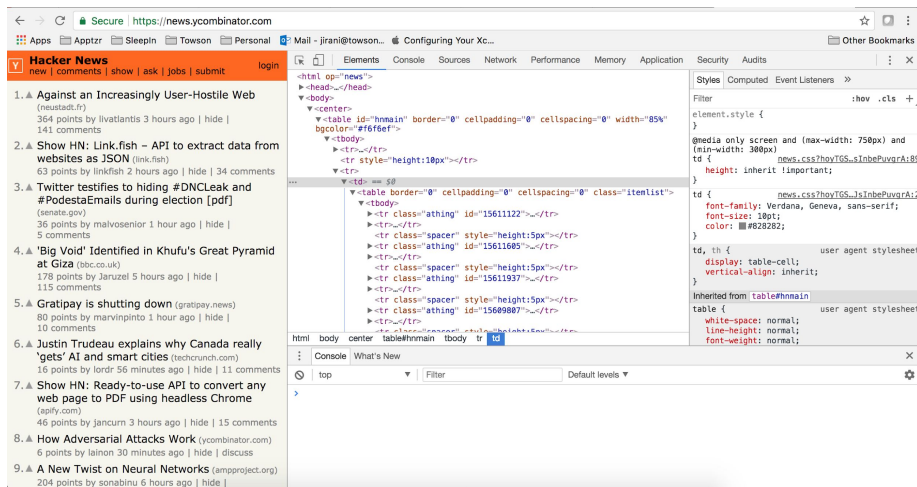
Building a Simple Web Scraper (cont.)

- Request from news.ycombinator.com (this is a GET)
- Ensure that the response code is a 200 - success
- Printing out the raw HTML

```
request('https://news.ycombinator.com', function (error, response, html) {  
    if (!error && response.statusCode == 200) {  
        console.log(html);  
    }  
});
```

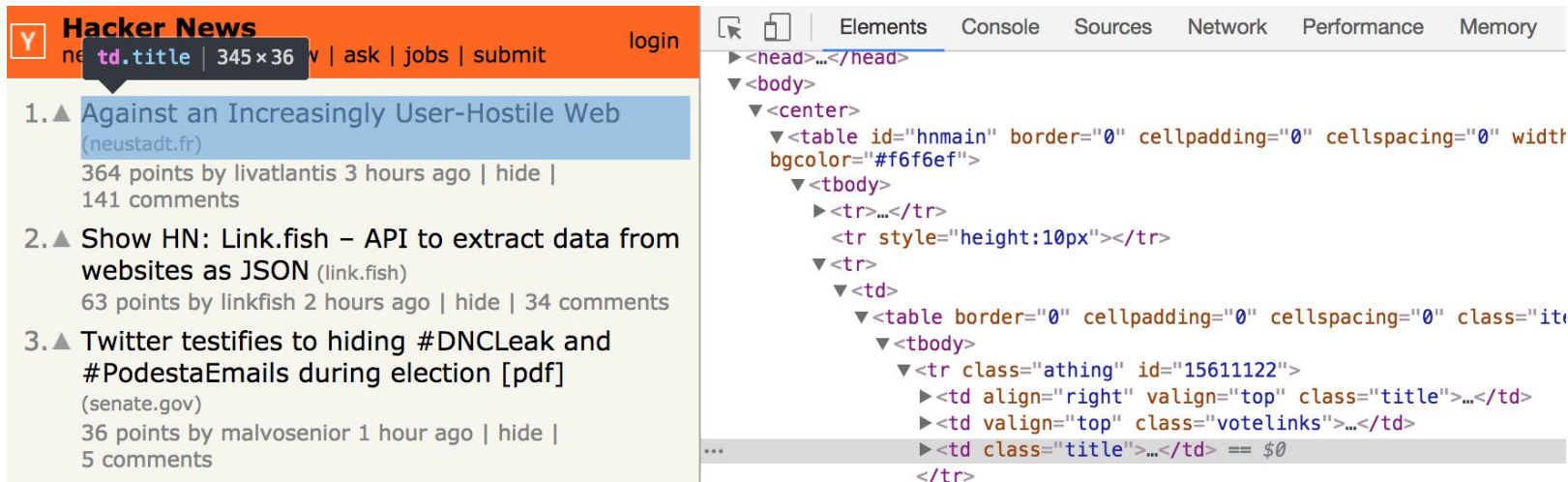
Building a Simple Web Scraper (cont.)

- To scrape an HTML page, it helps to know the anatomy of the page you will be scraping
- Right click->Inspect and view the HTML source by hovering over elements



Building a Simple Web Scraper (cont.)

- Let's pull the titles of the first 10 news articles



The image shows a screenshot of the Hacker News website on the left and its corresponding DOM structure on the right. The website header is orange with the 'Y' logo, 'Hacker News' text, a search bar, and navigation links. The main content area lists three news articles. The first article is highlighted with a blue selection box. The DOM on the right shows the HTML structure, with the 'Elements' panel expanded to show the body and table elements. The first article's title is highlighted in the DOM tree.

Hacker News ne **td.title** | 345x36 v | ask | jobs | submit login

- ▲ Against an Increasingly User-Hostile Web (neustadt.fr)
364 points by livatlantis 3 hours ago | hide | 141 comments
- ▲ Show HN: Link.fish – API to extract data from websites as JSON (link.fish)
63 points by linkfish 2 hours ago | hide | 34 comments
- ▲ Twitter testifies to hiding #DNCLeak and #PodestaEmails during election [pdf] (senate.gov)
36 points by malvosenior 1 hour ago | hide | 5 comments

DOM Structure (Elements panel):

```
<head>...</head>
<body>
  <center>
    <table id="hnmain" border="0" cellpadding="0" cellspacing="0" width="100%" bgcolor="#f6f6ef">
      <tbody>
        <tr>...</tr>
        <tr style="height:10px"></tr>
        <tr>
          <td>
            <table border="0" cellpadding="0" cellspacing="0" class="item">
              <tbody>
                <tr class="athing" id="15611122">
                  <td align="right" valign="top" class="title">...</td>
                  <td valign="top" class="votelinks">...</td>
                  <td class="title">...</td> == $0
                </tr>
              </tbody>
            </table>
          </td>
        </tr>
      </tbody>
    </table>
  </center>
</body>
```


Building a Simple Web Scraper (cont.)

- We will now load the buffer into the “\$” variable
 - Which will act as a selector for HTML elements

```
var $ = cheerio.load(html);
```

Building a Simple Web Scraper (cont.)

- We then want to loop through each td element that is part of the title class
 - Note we use each and not forEach, this is the jQuery version

```
// Not to be confused with forEach, this is jQuery
$('td.title').each(function(i, element) {

});
```

Building a Simple Web Scraper (cont.)

- Since we are inside of the `.each` using `$(this)` represents the current iterated DOM object
- Essentially we are printing the contents of the current iterated object
- `.text()` will print the corresponding text that is associated with the tag

```
// Not to be confused with forEach, this is jQuery
$('td.title').each(function(i, element) {
    console.log($(this).text());
});
```

Building a Simple Web Scraper (cont.)

```
Jals-MacBook-Pro:Lecture10 jolirani$ node simple-cheerio.js
1.
Savitech USB audio drivers install a new root CA certificate (cert.org)
2.
Against an Increasingly User-Hostile Web (neustadt.fr)
3.
USS McCain collision ultimately caused by UI confusion (arstechnica.co.uk)
4.
How Adversarial Attacks Work (ycombinator.com)
5.
Tor's Fall Harvest: The Next Generation of Onion Services (torproject.org)
6.
The Exercise Pill (newyorker.com)
7.
MySQL vs. MariaDB: Reality Check (percona.com)
8.
Show HN: Link.fish - API to extract data from websites as JSON (link.fish)
9.
'Big Void' Identified in Khufu's Great Pyramid at Giza (bbc.co.uk)
10.
BigchainDB - A scalable blockchain database (github.com)
11.
The freakishly strong base (collaborativefund.com)
12.
Apple Reports Fourth Quarter Results (apple.com)
13.
What I Learned From Losing $200M (2015) (nautil.us)
14.
Show HN: Ready-to-use API to convert any web page to PDF using headless Chrome (apify.com)
15.
A New Twist on Neural Networks (ampproject.org)
16.
Julian Assange confirms Cambridge Analytica sought WikiLeaks' help (theguardian.com)
17.
Gratipay is shutting down (gratipay.news)
18.
Linux containers in 500 lines of code (lizzie.io)
19.
Donate your TALENT to support your favorite charities (edgcoders.com)
20.
Show HN: SkyMind Intelligence Layer Community Edition (skymind.ai)
21.
Heart Stents Fail to Ease Chest Pain (nytimes.com)
22.
Unsupervised Machine Translation Using Monolingual Corpora Only [pdf] (arxiv.org)
23.
```

Building a Simple Web Scraper (cont.)

```
// Not to be confused with forEach, this is jQuery
$('td.title').each(function(i, element) {
    if (i % 2 == 1 && i < 20) {
        console.log($(this).text());
    }
});
```

Build Your Own

- Total up the points for all 30 news articles displayed on the first page

Node Projects

Node Projects

- So far in the class, you all have been creating single files and running them; fairly simple
- Let's say you wanted to make a Node.js server open-source so developers could actively contribute to them
- Can you think of a problem?
 - They would need to open all of your files and "npm install <package-name>" on all individual packages which would be quite annoying

Node Projects (cont.)

- To avoid this problem, you will use the “npm init” command
- Npm init will ask a few configuration questions:
 - Author name
 - Git repo
 - Version number
 - Etc
- You can omit almost all of the questions, but it is just initial setup

npm init

- Once configuration is complete, a file called package.json is created
- This is your configuration file (stored as JSON) and will help you manage your packages, versioning, licenses, etc.

```
{
  "name": "test",
  "version": "1.0.0",
  "description": "Example for AIT 618",
  "main": "index.js",
  "scripts": {
    "test": "None"
  },
  "author": "Jal Irani",
  "license": "ISC"
}
```

Installing Packages

- Now when you install a package use
 - `npm install <package-name> --save`
- The `--save` flag will automatically save the package to “package.json” and pin a version number to it
- Why is this important?
 - Say Fred makes a server that has a specific version of cheerio, request, and express
 - When Diana pulls his code from a repository, all she needs to run is “npm install” which will read the package.json file and install the correct packages and versions for those packages